

Agent Mesh SRE

Development Chronicle

A complete prompt-by-prompt record of how this application was built,
from first feature request to production deployment.

Stack: Next.js 14 · TypeScript · React · KafkaJS · Confluent for Kubernetes on Civo · Vercel

Repo: github.com/surajcsibm/agent-mesh-sre-real

Deployed: agent-mesh-sre.vercel.app · Kafka: 212.2.248.241:9092

Prompts documented: 38 major prompts across 2 sessions

How to Read This Document

This chronicle captures every significant developer prompt that shaped Agent Mesh SRE, in the order it was given. Each entry includes the prompt itself (paraphrased where spoken or typo-filled, but preserving exact intent), what was being changed or built at that moment, the technical outcome, and where relevant, the context needed to understand why this was the right move.

The document is written for a developer reading it cold — someone who might continue this project using Claude, ChatGPT, Copilot, or any other AI assistant, or simply work on it independently. Every section assumes working knowledge of Kafka, Kubernetes, and Next.js, but no prior knowledge of this specific codebase.

The application was built incrementally across two sessions. Session 1 covers the UI/UX feature cycle and the Kafka infrastructure migration. Session 2 covers the final UI polish, approval workflow completion, Civo cluster provisioning, and documentation. Both are captured here in full.

Application Overview

Agent Mesh SRE is a proof-of-concept platform for AI-governed Kafka operations. It implements a four-agent mesh — Intake, Monitor, Writer, and Notification — connected through Kafka topics, with a real-time animated dashboard, human-in-the-loop approval gates, and a self-improving lessons-learned cycle.

Core Architecture

- Framework: Next.js 14 App Router, TypeScript 5, React 18
- Real-time: Server-Sent Events (SSE) from `/api/mesh/stream` to the browser
- Kafka: KafkaJS — works against any Kafka provider (Aiven, Confluent, Strimzi)
- State: React state (client UI) + Kafka topics (persistent) + localStorage (lesson history)
- Auth: NextAuth v4 with Google OAuth
- Deploy: Vercel (auto-deploy on push to main)
- Kafka cluster: Confluent for Kubernetes (CFK) on Civo — KRaft mode, 3 brokers, 7 topics, RF=3

The Four Agents

Agent	Role	MCP Tools	Produces to
Intake	MCP Gateway	intake.submitOpsRequest	ops.requests.v1
Monitor	MRAL Coordinator	kafka.scaleConsumers, acknowledgeFailover, checkpointShareGroup	ops.incidents.v1, ops.actions.audit.v1, ops.lessons.v1
Writer	Postmortem Author	writer.draftIncidentReport	ops.actions.audit.v1
Notification	Outbound Router	notify.slack, itsm.openTicket	ops.notifications.v1

The Seven Kafka Topics

Topic	Parts	Retention	Clean up	Purpose
ops.requests.v1	6	7d	delete	Operator requests from Intake agent
ops.kafka.metrics.v1	6	1d	delete	Cluster telemetry polled by Monitor
ops.incidents.v1	6	30d	delete	Incidents raised by Monitor agent
ops.actions.audit.v1	12	365d	delete	Full audit trail — every action and decision
ops.lessons.v1	3	inf	compact	Lessons learned — log-compacted, always available
ops.notifications.v1	3	7d	delete	Outbound alerts (Slack, ITSM, email)
demo.payments.events	24	3d	delete	Demo workload — high partition count for lag simulation

Session 1: UI/UX Feature Cycle

The first development session focused entirely on the Dashboard UI — specifically the approval workflow, scenario history, lessons learned panel, and the Kafka infrastructure migration from Aiven to Confluent for Kubernetes. The application was already scaffolded with a working simulator and SSE stream; this session refined the human-facing features and replaced the cloud Kafka dependency.

Phase 1: Approval Gate Redesign

Prompt 01 · Review popup before approve/reject

The original implementation showed Approve and Reject buttons directly on pending approval cards in the right panel. The request was to show only a single 'Review' button there, and open a full popup modal when clicked — so the operator could review all available details before committing to a decision. The modal should show: MCP tool call details, parameters, rationale, infrastructure mutation risk level, and Approve/Reject buttons at the bottom.

What changed: The ApprovalQueue panel was refactored to render only a 'Review & Decide' button per pending entry. Clicking it set a reviewingApproval state variable in the Dashboard component, which conditionally rendered the full ApprovalGate modal.

Prompt 02 · Not working — still see approve/reject on the right panel

The change didn't deploy correctly. Debugging involved grepping Dashboard.tsx and ApprovalQueue.tsx for old button patterns. Issue: changes were applied to the output file but the committed version still had the old pattern. Fix: re-patched and pushed to git.

Prompt 03 · Add trigger reasons + close button to the approval popup

Two additions: (1) Show the reason why the failure was triggered in the approval popup, drawn from a pre-defined list of per-scenario trigger conditions. (2) Add a close button (top-left X) so the operator can dismiss the popup without making a decision. SCENARIO_TRIGGER_REASONS was added as a module-level constant — a map from scenario ID to an array of candidate trigger reasons. The X button set reviewingApproval to null.

Prompt 04 · Show scenario trigger reasons from attached condition list

A detailed list of trigger conditions was provided for all 11 scenarios: lag spike, controller failover, share group rebalance, schema mismatch, disk saturation, under-replication, producer timeout, consumer session timeout, compaction lag, partition imbalance, and benign rebalance. These formed the canonical SCENARIO_TRIGGER_REASONS map.

Prompt 05 · Deploy to Vercel

First consolidated deploy of the approval gate changes.

Prompt 06 · Show only the single most probable trigger reason

The popup was showing all trigger reasons. Simplified to show only the single most likely reason — the one with highest confidence for that specific scenario.

Prompt 07 · Change to top two most likely reasons

One reason felt too sparse. Settled on showing the top two most likely triggers — a balance between brevity and context for the operator.

Prompt 08 · Remove the toast notification for approvals

An amber banner at the bottom of the screen was firing when approvals appeared. It was distracting and redundant given the right-panel indicator. Removed entirely.

Phase 2: Scrollable Modals and Lessons Learned

Prompt 09 · Make scenario detail popups scrollable and viewport-fitted

The ScenarioEndModal (shown when clicking a history entry) was rendering off-screen on smaller viewports. Request: constrain popup height to the browser window and make the body scrollable if content overflows, while keeping the header and footer sticky. Fix: modal outer div got `max-h-[calc(100vh-2rem)]` with `overflow-hidden`. Header got `flexShrink:0`. Body section got `overflowY:auto` and `flex-1`.

Prompt 10 · Add lessons learned to the scenario detail popup

When clicking a scenario from the history bar, the popup should show the lesson recorded for that run at the bottom — the Monitor agent's structured learning output including action taken, effectiveness rating, lag delta, adjusted threshold, and notes. Also: clicking a lesson card in the bottom-right panel should open a Lesson Detail modal. LessonDetailModal was added as a new component. A lessonHistory state array was added, backed by localStorage under the key 'sre-lesson-history', holding up to 100 entries across all sessions.

Prompt 11 · Lessons learned not showing in the popup or the right panel

After the initial implementation, lessons weren't appearing. Debug traced the issue to the hydration timing — the lesson was recorded server-side via SSE (Server-Sent Events), but the client-side emailSummary object used for history wasn't picking it up. Synchronized SSE lesson events with client state properly.

Prompt 12 · Lessons learned history is being overwritten — keep all lessons across scenarios

When a second scenario fired, its lesson replaced the first. The request was for a persistent accumulating list — newest at top, up to 100 entries, surviving page refresh, each entry clickable to show full details. Fix: a useEffect watching state.lessons from the SSE stream deduped new entries by ID against the lessonHistory array in localStorage. Each new lesson was prepended. The sidebar showed the full list with clickable cards, each opening LessonDetailModal.

Phase 3: Kafka Infrastructure Migration — Aiven to CFK

Prompt 13 · What changes are needed to move from Aiven to Confluent for Kubernetes?

The existing application connected to an Aiven managed Kafka cluster using Aiven's proprietary REST API (aiven-admin.ts) and SCRAM-SHA-256 + TLS. Goal: migrate to self-managed Confluent for Kubernetes (CFK) cluster, removing the Aiven dependency entirely. Analysis identified: aiven-admin.ts, mesh.ts, monitor-poll.ts, and admin API routes as the key files to change. Plan: create kafka-admin-cfk.ts — pure KafkaJS, provider-agnostic, working against any Kafka cluster.

**Prompt
14****· Let's go from the beginning — step by step migration to CFK on-prem**

Full migration walkthrough requested: (1) Add Confluent Helm repo and create confluent namespace. (2) Install CFK operator via Helm with `namespaced=false`. (3) Apply `confluent-platform.yaml` manifest defining KRaftController (3 replicas), Kafka (3 brokers), SchemaRegistry, and KafkaRestClass. (4) Create all 7 required topics inside the cluster. (5) Update environment variables and replace `aiven-admin.ts` with `kafka-admin-cfk.ts`.

**Prompt
15****· Helm and kubectl commands — operator install**

Docker Desktop was the local Kubernetes cluster. Issues encountered: namespace didn't exist before Helm install (created separately), Docker Desktop connection was refused (app wasn't running), port 9092 was already in use (switched to port-forward on 19092). All resolved, CFK operator and cluster installed successfully.

**Prompt
16****· Topics created — now expose the cluster externally**

All 7 topics were created inside the cluster via `kubectl exec`. Challenge: port-forward only exposes Kafka to localhost — Vercel can't reach it. Options explored and rejected: NodePort (headless service constraint), Cloudflare Tunnel (requires payment card for TCP), ngrok (requires payment card for TCP), localhost.run (SSH tunnel — unstable). Conclusion: locally port-forwarded Kafka cannot serve a globally deployed Vercel app.

**Prompt
17****· Need a free cloud Kafka provider accessible to everyone**

Decision: move to Civo — a Kubernetes cloud with a generous free tier, stable connection, no credit card requirement, and a permanent IP address. `kubernetes-native`, so CFK runs cleanly there.

**Prompt
18****· Civo is better — let's use that**

Civo selected. Account verification was pending, so `kafka-admin-cfk.ts` was written in parallel. The module implements: `listTopics`, `getTopicDetails`, `getConsumerGroups`, `createTopic`, `deleteTopic`, `updateRetention` — all via pure KafkaJS, no proprietary Aiven API calls. All import references in `mesh.ts`, `monitor-poll.ts`, and admin API routes were updated. `aiven-admin.ts` was deleted from the repository.

Prompt 19 · Civo account active — set up CFK cluster

Cluster created: 3-node K3s, g4s.kube.medium (2 CPU, 4GB). CFK operator installed. Kafka 8.2.0 KRaft mode cluster provisioned with fsGroup:1000 / runAsUser:1000 pod security context for K3s permissions. PVCs set to 10Gi per broker (100Gi hit Civo's free tier limit). After resolving PVC Pending states, all 3 brokers came up Running. External listener configured via CFK's loadBalancer external access type with nip.io domain. External endpoint: 212.2.248.241:9092. All 7 topics created at RF=3. Connection test confirmed from local machine.

Phase 4: UI Label Cleanup Post-Migration

Prompt 20 · Still shows Aiven in the UI — update all references to CFK

After the infrastructure migration, the Dashboard still displayed 'AIVEN · DIRECT KAFKA'. Multiple grep passes identified references in Dashboard.tsx, TopBar.tsx, and cluster-status.ts. TopBar.tsx: 'Aiven Kafka · Connected' became 'CFK · Confluent for Kubernetes'. Dashboard.tsx ClusterStatsModal header updated. The isAiven boolean was repurposed as isDirectKafka.

Prompt 21 · Bootstrap URL not visible in the Cluster Statistics panel

The bootstrap host was being truncated — the code was taking only the first two parts of the hostname and appending '...'. Fix: const hostShort = host (no truncation). The full IP address 212.2.248.241 now displays correctly.

Prompt 22 · Auth User and CA Certificate values are wrong in the real mode popup

The panel was showing hardcoded 'avnadmin' and 'SCRAM-SHA-256' regardless of what the environment variables contained. The CFK cluster runs without authentication. Fixes: Auth User changed to kafka.username || '(no auth)'. SASL Mechanism changed to kafka.saslMechanism || '(none)'. CA Certificate changed to show 'Not required' when hasCaCert is false. saslMechanism field added to ModelInfo.kafka type in cluster-status.ts and to publicView() in runtime-mode.ts.

**Prompt
23****· Create a migration writeup and user guide**

Documentation generated: CFK-Migration-Writeup.docx, CFK-Migration-Summary.docx, and Agent-Mesh-SRE-User-Guide-v3.2.docx. All pushed to public/docs/ in the repository, accessible at agent-mesh-sre.vercel.app/docs/.

**Prompt
24****· Clean up aiven-admin.ts — remove it**

With kafka-admin-cfk.ts fully replacing it, aiven-admin.ts was deleted via `git rm` and committed. The repository no longer contains any Aiven-specific code.

Session 2: Final UI Polish and Civo Deployment

The second session picked up after the Aiven migration. It focused on completing the UI feature set: scenario timing control, root-cause intelligence in all popups, approval workflow improvements, and a full documentation pass.

Phase 5: Scenario Timing and Root Cause Intelligence

Prompt 25 · Scenarios firing too fast — add minimum gap between auto-triggers

The anomaly simulation on Vercel was firing scenarios every 15-25 seconds, causing multiple scenarios to run in parallel. Request: minimum 30-40 second gap between automatic scenario fires, with a hard 10-second minimum guard inside runCycle. anomaly-sim.ts updated: FIRST_DELAY to 20s/35s, MIN_INTERVAL to 30s/90s, MAX_INTERVAL to 40s/150s (Vercel/local). Hard MIN_GAP_MS guard prevents any two scenarios within 10 seconds.

Prompt 26 · Show all possible root causes + most probable in history popup and approval gate

When clicking a scenario in the history bar, the popup should show: (1) a red-bordered card with the single most probable root cause, and (2) an orange-bordered list of all four candidate causes with the most probable highlighted in bold red. Same sections in the approval gate popup. SCENARIO_TRIGGER_REASONS updated from old shape (title + reasons: [string, string]) to new shape (mostLikely: string, allReasons: string[]). SCENARIO_LABEL_TO_ID added to map scenario display labels to scenario IDs, since EmailSummaryData uses scenarioLabel not scenarioId.

Prompt 27 · Rename labels: Root Cause and Available Root Causes

Section headers renamed: 'Most Probable Root Cause' became '■ Root Cause'. 'All Possible Root Cause Reasons' became '■■ Available Root Causes'. Applied in both the approval gate and the scenario history popup.

Phase 6: Pause/Resume Auto-Trigger and Manual/Auto Badges

Prompt 28 · Pause/resume, manual/auto labels, and Escape key to close modals

Three features requested together: (1) A Pause/Resume button at the top of the left scenario panel — when paused, SSE-triggered autonomous scenarios don't fire, but operator-clicked scenarios always do. (2) A ■ Manual / ■ Auto badge on every scenario history entry. (3) Pressing Escape closes any open modal. Implementation: `simPaused` state added; `useEffect` synced it to `window.__simPaused`; `useMeshStream.ts` checks `window.__simPaused` before calling `runClientScenario` on SSE auto-trigger events; `lastManualTriggerRef` tracked manually clicked scenario IDs; history entries stamped with `triggerSource`. Escape handler closes all modal states.

Phase 7: Approval Workflow Completion

Prompt 29 · Review button and Send for Approval button for awaiting-approval history entries

Scenario history entries with status 'Awaiting Approval' should have an amber 'Review →' button that opens the approval gate directly. The scenario detail popup for those entries should have a 'Send for Approval →' button at the bottom. `ScenarioHistoryBar` was updated to accept `onReview` and `pendingApprovals` as props. Review button synthesised an `ApprovalRequest` from the history entry using `SCENARIO_TOOL_CALLS` — a new map from scenario ID to the canonical MCP tool call details — so the approval gate renders with full rich detail rather than the generic fallback. Send for Approval button adds to `localPendingApprovals` and opens the gate.

Prompt 30 · Approve/Reject decisions must update the history entry status

When an operator approves or rejects a synthesised approval, the corresponding scenario history entry should update its badge from ■ Awaiting Approval to ✓ Resolved or ✗ Rejected. The `onDecide` handler was updated to call `setSummaryHistory`, mapping over entries and finding the one matching the approval's `scenarioId`, then updating its `approved` and `status` fields. Entries also removed from `localPendingApprovals` on decision.

Prompt 31 · Root causes not showing for rejected entries

Rejected scenario entries were opening the detail popup but not showing Root Cause sections. The trigger reasons block had been lost in a subsequent patch. Fix: the IIFE trigger reasons block was re-inserted between the Monitor→Reason section and the Act section in ScenarioEndModal. Uses scenarioId prop (not SCENARIO_LABEL_TO_ID[data.scenarioLabel]) to look up the correct reason set, so it works for all entries regardless of scenarioLabel format.

Phase 8: Cluster Statistics Panel Updates

Prompt 32 · Show the internal Kubernetes URL in broker info

The Cluster Statistics panel showed only the Bootstrap Host (external IP) but not the internal K8s DNS address. A new row was added: Internal (K8s) = kafka.confluent.svc.cluster.local:9092.

Prompt 33 · SASL mechanism and auth user still showing old Aiven values

The Cluster Statistics panel was showing SCRAM-SHA-256 and avnadmin even though those environment variables were cleared. Root cause: saslMechanism was defined in RuntimeConfig but not included in the publicView() return object. Fix: added saslMechanism to publicView() in runtime-mode.ts and to the ModelInfo.kafka type in cluster-status.ts.

Phase 9: Civo Cluster Full Provisioning

Prompt 34 · Civo is active — set up CFK cluster from scratch

Steps: (1) Create 3-node K3s cluster 'agent-mesh-kafka', download kubeconfig to ~/cfk/civo-kubeconfig.yaml. (2) Install CFK operator via Helm. (3) Apply confluent-platform.yaml with fsGroup:1000 / runAsUser:1000 pod security context. PVC provisioning required 10Gi per broker (100Gi hit free tier limit). kafka-1 PVC got stuck with storageClass: dummy — resolved by deleting all Kafka CRs, PVCs, and pods and reapplying. External listener configured via CFK loadBalancer type with nip.io domain; per-broker LoadBalancer services provisioned. Broker advertised listener issue resolved by pod restart. Connection confirmed: 14 topics visible (7 app + 7 internal Confluent). All 7 required topics created at RF=3.

Prompt 35 · Enable SCRAM authentication on CFK cluster

Attempted to add SCRAM authentication. CFK does not support 'scram' as a listener authentication type — only plain, digest, mtls, ldap, oauth, bearer, none. Attempted plain auth instead; broker pods crashed due to incorrect JAAS secret format. SASL deferred — cluster runs without auth for the demo. To add auth in future: use plain listener with the correct Confluent JAAS format, or use mTLS.

Phase 10: Documentation

Prompt 36 · Create latest user guide

User Guide v4.0 generated covering: MRAL loop, all 4 agents, scenario history bar with badges, scenario detail popup (including Root Cause and Available Root Causes for all entry types including rejected), approval workflow (3 entry points: auto gate, Review →, Send for Approval →), lessons learned persistent history, Civo cluster details, environment variables, keyboard shortcuts, troubleshooting.

Prompt 37 · Create a Kafka developer/admin presentation — 10 slides

Professional PowerPoint presentation created: The Challenge, Meet Ken & Ben (Kafka admin and developer personas), MRAL loop architecture, Four Agents, Seven Topic Design, 11 Scenarios, Approval Workflow, Root Cause Analysis example, Audit and Lessons, Technical Stack, Closing summary. Light blue professional palette, Cambria headers, Arial body. Speaker notes on every slide. Slide 2 specifically frames the POC from the perspective of two real Kafka users.

Prompt 38 · SSE needs to be spelled out — and tone down the boasting

The presentation was redesigned: lighter blue palette, SSE expanded to 'Server-Sent Events (SSE)' with explanation in speaker notes, copy rewritten to be factual and restrained ('current workflow:' not 'pain points:', closing slide says 'What This POC Demonstrates'). Zustand replaced with 'React state (client UI)' in the tech stack slide.

Key Technical Decisions and Learnings

1. SCENARIO_LABEL_TO_ID — Why It Was Needed

EmailSummaryData (the object stored in scenario history) has a scenarioLabel field ('Consumer Lag Spike', 'Lag Spike', etc.) not a scenarioId field ('lag-spike'). SCENARIO_TRIGGER_REASONS and SCENARIO_TOOL_CALLS are keyed by scenario ID. A mapping constant was required to bridge the two. Over time, multiple label variants were added because the client simulator uses shorter labels than the left sidebar button definitions.

Lesson: When building event-driven UIs where the same concept appears under multiple string identities across components, define a canonical ID early and map everything to it. Retrofitting mapping later caused multiple build failures and ReferenceErrors.

2. Component Scope and Module-Level Constants

SCENARIO_LABEL_TO_ID and SCENARIO_TRIGGER_REASONS are defined at module level in Dashboard.tsx outside the component function. ScenarioEndModal is also defined at module level as a separate function — not inside Dashboard. It cannot access module-level constants defined after it in the file.

The fix: pass scenarioId as a prop from the Dashboard component (which has access to the constants) to ScenarioEndModal, rather than having the modal look up the ID itself. This pattern — pushing computed values down as props rather than having child components reach up for globals — applies throughout the codebase.

3. CFK Kafka External Listener Pattern

The core issue: Kafka brokers advertise their own hostnames/IPs in metadata responses. A client connecting to the bootstrap IP receives broker addresses in the metadata response, and if those addresses are internal K8s DNS names, the client cannot reconnect to the actual brokers.

Solution: CFK's external access type loadBalancer with a domain using the nip.io wildcard DNS service. Each broker gets its own LoadBalancer IP (kafka-0-lb, kafka-1-lb, kafka-2-lb) and advertises that IP:port externally. The bootstrap LoadBalancer (kafka-bootstrap-lb) routes initial connections.

4. React State Batching and Modal Transitions

When ScenarioEndModal closes and sets reviewingApproval in the same render cycle, React may batch the updates such that the ApprovalGate never appears. The working pattern: the onSendForApproval prop sets localPendingApprovals and reviewingApproval first, then calls onClose. The ApprovalGate is rendered after ScenarioEndModal in the JSX tree, so it appears after the modal unmounts.

5. Patch Workflow and File Version Drift

A recurring failure mode: a file is uploaded (current disk version), patches are applied to the uploaded file and written to an output file, the user commits the output, but then a later session continues from a new

upload that is ahead of what was patched. This causes some patches to be applied twice (duplicating sections) and others to be missing.

The reliable pattern: run a grep audit before every patch to confirm exactly what is present in the current disk file. Use Python scripts that search for exact string anchors (not line numbers, which shift). Verify with a checklist after every patch before committing.

Current Application State

What Is Working

- Kafka cluster: Confluent for Kubernetes on Civo, 3 brokers, 7 topics at RF=3, external IP 212.2.248.241:9092
- All 11 scenarios run in simulator mode — 4 require human approval, 7 auto-remediate
- Scenario history bar: all runs shown with ✓ Resolved / ■ Awaiting Approval / ✗ Rejected badges and ■ Manual / ■ Auto trigger source badges
- Scenario detail popup: Root Cause (red card) + Available Root Causes (ranked list) for all entries including rejected ones
- Approval gate: shows tool call, parameters, rationale, Root Cause, Available Root Causes; triggered from auto SSE, Review → button, or Send for Approval → button
- Approval decisions update the corresponding history entry badge in real time
- Pause/Resume auto-trigger button in scenario panel; paused state synced to `window.__simPaused`
- Lessons learned: persistent `localStorage` history up to 100 entries, clickable detail modals
- Cluster Statistics panel: external bootstrap IP, internal K8s DNS, SASL mechanism from env, correct auth user and CA cert status
- Escape key closes all modals: scenario history, approval gate, lesson detail, broker info, cluster stats, topic modal, create topic
- User Guide v4.0, Migration Writeup, and Kafka Developer Presentation deployed to `public/docs/`

Known Limitations

- CFK cluster has no authentication — runs open for demo purposes. To add auth: use plain listener with correct Confluent JAAS format, or use mTLS.
- Scenario timing (30–40 seconds) works on Vercel; local dev may fire faster. `window.__simPaused` only pauses auto-trigger, not manually clicked scenarios.
- `SCENARIO_LABEL_TO_ID` covers known label variants but `client-sim.ts` may produce label strings not in the map for new scenarios — root cause lookup will fall back to an empty string and show no reasons.
- The Send for Approval → button appears only for entries where `data.action` contains 'awaiting' or 'approval'. New scenario types with different action strings may not trigger it.

Environment Variables (Production on Vercel)

Variable	Value / Notes
KAFKA_MODE	real
KAFKA_BOOTSTRAP	212.2.248.241:9092
KAFKA_SASL_MECHANISM	(empty — no auth on demo cluster)
KAFKA_USERNAME	(empty)
KAFKA_PASSWORD	(empty)
KAFKA_CA_CERT_BASE64	(empty — no TLS on demo cluster)
NEXTAUTH_SECRET	32-byte random — mark as Secret in Vercel
NEXTAUTH_URL	https://agent-mesh-sre.vercel.app
GOOGLE_CLIENT_ID	From Google Cloud Console
GOOGLE_CLIENT_SECRET	From Google Cloud Console — mark as Secret

Git Repositories

- Primary (real-ops): github.com/surajcsibm/agent-mesh-sre-real — branch: main
- Public demo: github.com/surajcsibm/agent-mesh-sre
- PR target: github.com/aswinayyolath/agent-mesh-sre
- Deployed: agent-mesh-sre.vercel.app (auth-gated via Google OAuth)
- Local dev: localhost:3002 (real-ops) / localhost:3000 (original)
- Civo kubeconfig: ~/cfk/civo-kubeconfig.yaml — export KUBECONFIG=~/cfk/civo-kubeconfig.yaml

Guide for Continuing Development

When Adding a New Scenario

1. Add the scenario definition to PINNED_SCENARIOS or EXTRA_SCENARIOS in Dashboard.tsx with id, label, badge, color
2. Add the scenario ID to SCENARIO_LABEL_TO_ID with all expected label variants
3. Add the scenario entry to SCENARIO_TRIGGER_REASONS with mostLikely and allReasons (4 candidates)
4. Add the scenario entry to SCENARIO_TOOL_CALLS with the canonical MCP tool name and arguments
5. Add to SCENARIO_AUTO_TOPICS if it requires specific topics to be auto-created
6. Add the scenario handler to anomaly-sim.ts
7. Register the tool call in mcp.ts in the MCP_TOOLS registry

When Patching Dashboard.tsx

- Always grep the current disk file first — never assume the uploaded file is current
- Search by exact string anchor, not line number — the file shifts significantly between sessions
- Run the audit checklist before committing — verify all critical constants, render blocks, and props are intact
- SCENARIO_LABEL_TO_ID must be defined before ScenarioEndModal in the file (module-level constant)
- ScenarioEndModal receives scenarioId as a prop — never reference SCENARIO_LABEL_TO_ID inside the component

When Changing the Kafka Cluster

- Update KAFKA_BOOTSTRAP in Vercel environment variables and in .env.local
- If adding auth: set KAFKA_SASL_MECHANISM, KAFKA_USERNAME, KAFKA_PASSWORD; add saslMechanism to publicView() in runtime-mode.ts if not already done
- If using custom CA: base64-encode the CA PEM and set KAFKA_CA_CERT_BASE64 (no line breaks on Vercel)
- Create all 7 topics before switching KAFKA_MODE to real — the app does not auto-create topics
- Test connection locally with node test-kafka.js before deploying to Vercel

The kubeconfig at ~/cfk/civo-kubeconfig.yaml must be exported before running kubectl against the Civo cluster: export KUBECONFIG=~/cfk/civo-kubeconfig.yaml. Without this it defaults to the local Docker Desktop cluster.